

Анализ данных: экзаменационный конспект

Теория, формулы, pandas и sklearn без прямых ответов из демо-заданий

Учебная шпаргалка для экзамена

2026-05-28

Содержание

1 Как пользоваться	3
2 Вероятность и распределения	3
2.1 Базовые правила	3
2.2 Формула полной вероятности и Байес	4
2.3 Математическое ожидание и дисперсия	4
2.4 Равномерное распределение	4
2.5 Бернулли, биномиальное и геометрическое распределения	5
2.6 Экспоненциальное распределение	5
2.7 Нормальное распределение и z-таблица	6
2.8 PDF и CDF	6
3 Статистика и проверка гипотез	6
3.1 Выборка, генеральная совокупность, признаки	6
3.2 Меры центра и разброса	7
3.3 Корреляция Пирсона и Спирмена	7
3.4 Нулевая гипотеза, p-value, ошибки	8
4 Визуализация	8
4.1 Выбор графика	8
4.2 Boxplot и violinplot	8
4.3 Bubble plot	8
5 Работа с данными в pandas	8
5.1 Первичный осмотр	8
5.2 Фильтрация и выбор столбцов	9
5.3 Пропуски	9
5.4 Создание и изменение столбцов	10
5.5 Агрегации и сводные таблицы	10
5.6 Корреляции	10
5.7 Merge, concat, сортировка	10
6 Кодирование категориальных признаков	11
6.1 Label encoding	11
6.2 One-hot encoding	11
7 Машинное обучение: базовая схема	11
7.1 Объекты, признаки, цель	11
7.2 Train/test split	11
7.3 Pipeline	12
8 Метрики регрессии	12
8.1 MAE	12
8.2 MSE и RMSE	12
8.3 MAPE и MSLE	12
8.4 R^2	13

9	Метрики классификации	13
9.1	Confusion matrix	13
9.2	Accuracy	13
9.3	Precision, recall, F1	13
9.4	ROC-AUC	14
9.5	PR-AUC	14
9.6	Выбор метрики по ситуации	14
10	Переобучение, недообучение, регуляризация	14
10.1	Недообучение	14
10.2	Переобучение	14
10.3	Когда нельзя сделать вывод	15
10.4	Регуляризация L1/L2	15
11	Кросс-валидация и подбор гиперпараметров	15
12	Линейная алгебра и ядра	16
12.1	Векторы, нормы, скалярное произведение	16
12.2	Kernel trick	16
13	Линейные модели и градиентный спуск	16
13.1	Линейная и логистическая регрессия	16
13.2	Градиентный спуск	17
13.3	Масштабирование и градиентный спуск	17
13.4	Momentum и AdaGrad	17
14	Деревья решений и ансамбли	18
14.1	Gini impurity	18
14.2	Information Gain	18
14.3	Random Forest	18
14.4	Gradient Boosting	18
15	Bias-Variance-Noise	18
16	Кластеризация и аномалии	19
16.1	K-means	19
16.2	DBSCAN	19
16.3	Агломеративная кластеризация	20
16.4	Спектральная кластеризация	20
16.5	Поиск аномалий	20
17	Тексты	20
17.1	Токенизация, стоп-слова, лемматизация	20
17.2	Bag of Words и TF-IDF	20
17.3	Снижение размерности текстовых признаков	21
18	Изображения	21
19	Нейронные сети	21
20	Быстрые sklearn-шаблоны	22
20.1	Регрессия	22
20.2	Классификация	22
20.3	Pipeline с preprocessing	23
21	Экзаменационные правила-ловушки	23
22	Краткий словарь	23

1 Как пользоваться

Документ предназначен как легальная справка: здесь нет готовых ответов на конкретные скриншоты или демо-вопросы, только определения, формулы, интерпретации и шаблоны кода. На экзамене сначала определяй тип задачи: вероятность, статистика, работа с таблицей, регрессия, классификация, кластеризация, текст, изображения или нейросети.

Поиск через Ctrl+F: Bayes, p-value, ошибка первого рода, Spearman, ROC-AUC, F1, MAE, RMSE, Gini, Information Gain, gradient descent, Lasso, scaling, bias, variance, k-means, DBSCAN, groupby, GridSearchCV, Pipeline, label encoding, лемматизация.

Мини-карта задач:

Ситуация	Тип задачи	Что обычно нужно
Предсказать число: цену, длительность, количество	Регрессия	MAE, MSE, RMSE, R^2
Предсказать класс: болен/здоров, мошенник/не мошенник	Классификация	confusion matrix, precision, recall, F1, ROC-AUC, PR-AUC
Найти группы без заранее известных меток	Кластеризация	k-means, DBSCAN, агломеративная кластеризация
Найти странные объекты, ботов, сбои, редкие паттерны	Поиск аномалий	anomaly detection, isolation forest, one-class models
Есть признаки X и ответы y	Обучение с учителем	regression/classification
Есть только X , ответов y нет	Обучение без учителя	clustering, dimensionality reduction

2 Вероятность и распределения

2.1 Базовые правила

Вероятность события A лежит в пределах:

$$0 \leq P(A) \leq 1.$$

Дополнение:

$$P(\bar{A}) = 1 - P(A).$$

Если $A \subset B$, то:

$$P(A) \leq P(B).$$

Объединение событий:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Если события несовместны:

$$P(A \cup B) = P(A) + P(B).$$

Условная вероятность:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Независимость:

$$P(A \cap B) = P(A)P(B).$$

Важно: независимые события и несовместные события - разные понятия. Два несовместных события с положительными вероятностями не являются независимыми.

2.2 Формула полной вероятности и Байес

Если гипотезы $H_1, \dots, H_n$ образуют полную группу, то:

$$P(A) = \sum_{i=1}^n P(A | H_i)P(H_i).$$

Формула Байеса:

$$P(H_k | A) = \frac{P(A | H_k)P(H_k)}{\sum_{i=1}^n P(A | H_i)P(H_i)}.$$

Шаблон решения:

1. Выписать гипотезы H_i .
2. Выписать априорные вероятности $P(H_i)$.
3. Выписать условные вероятности $P(A | H_i)$.
4. Посчитать $P(A)$ по формуле полной вероятности.
5. Подставить в формулу Байеса.

2.3 Математическое ожидание и дисперсия

Для дискретной случайной величины:

$$E(X) = \sum_i x_i p_i.$$

Дисперсия:

$$D(X) = E[(X - E(X))^2] = E(X^2) - [E(X)]^2.$$

Стандартное отклонение:

$$\sigma(X) = \sqrt{D(X)}.$$

Свойства:

$$E(aX + b) = aE(X) + b,$$

$$D(aX + b) = a^2 D(X).$$

Если X и Y независимы:

$$D(X + Y) = D(X) + D(Y).$$

2.4 Равномерное распределение

Если $X \sim U[a, b]$, то все значения на отрезке $[a, b]$ равновероятны.

Плотность:

$$f(x) = \frac{1}{b-a}, \quad x \in [a, b].$$

Математическое ожидание:

$$E(X) = \frac{a+b}{2}.$$

Дисперсия:

$$D(X) = \frac{(b-a)^2}{12}.$$

Если задача спрашивает разброс времени/нагрузки при равномерном распределении на отрезке, почти всегда нужна именно формула $D(X)$ или стандартное отклонение $\sqrt{D(X)}$.

2.5 Бернулли, биномиальное и геометрическое распределения

Бернулли: $X \sim \text{Bernoulli}(p)$, один успех или неуспех.

$$E(X) = p, \quad D(X) = p(1-p).$$

Биномиальное распределение: $X \sim \text{Bin}(n, p)$ - количество успехов в n независимых испытаниях.

$$P(X = k) = C_n^k p^k (1-p)^{n-k}.$$

$$E(X) = np, \quad D(X) = np(1-p).$$

Геометрическое распределение: $X \sim \text{Geom}(p)$ - номер попытки, на которой впервые произошёл успех.

$$P(X = k) = (1-p)^{k-1} p, \quad E(X) = \frac{1}{p}.$$

Маркер геометрического распределения: «попытки независимы», «вероятность успеха постоянна», «до первого успеха».

2.6 Экспоненциальное распределение

$X \sim \text{Exp}(\lambda)$ часто описывает время ожидания до события.

Плотность:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Функция распределения:

$$F(x) = P(X \leq x) = 1 - e^{-\lambda x}.$$

Хвост:

$$P(X \geq x) = e^{-\lambda x}.$$

Среднее и дисперсия:

$$E(X) = \frac{1}{\lambda}, \quad D(X) = \frac{1}{\lambda^2}.$$

Экспоненциальное распределение имеет бесконечный правый хвост. Нельзя утверждать, что событие точно произойдет не позже фиксированного времени; можно только говорить, что вероятность после большого времени мала.

Полезное сравнение событий: если $a > b$, то событие $X \geq a$ уже, чем $X \geq b$, поэтому $P(X \geq a) \leq P(X \geq b)$.

2.7 Нормальное распределение и z-таблица

Если $X \sim N(\mu, \sigma^2)$, то стандартизация:

$$Z = \frac{X - \mu}{\sigma}, \quad Z \sim N(0, 1).$$

После этого вероятности ищутся по таблице стандартного нормального распределения.

Некоторые таблицы дают $P(Z \leq z)$, а некоторые дают площадь между 0 и z :

$$P(0 \leq Z \leq z).$$

Если таблица дает площадь между 0 и положительным z , то:

$$P(Z \leq z) = 0.5 + P(0 \leq Z \leq z),$$

$$P(Z \geq z) = 0.5 - P(0 \leq Z \leq z).$$

Для отрицательных значений используется симметрия:

$$P(Z \leq -z) = P(Z \geq z).$$

2.8 PDF и CDF

Плотность распределения, или PDF, обычно обозначается $f(x)$. Она показывает, где вероятность сконцентрирована сильнее. Плотность может расти и убывать, иметь несколько пиков, а ее значения могут быть больше 1; важна площадь под графиком.

Функция распределения, или CDF:

$$F(x) = P(X \leq x).$$

Свойства CDF:

- $0 \leq F(x) \leq 1$;
- $F(x)$ не убывает;
- при движении вправо стремится к 1;
- вероятность на интервале: $P(a < X \leq b) = F(b) - F(a)$.

Мода непрерывного распределения находится по максимуму плотности, а не по максимуму функции распределения.

3 Статистика и проверка гипотез

3.1 Выборка, генеральная совокупность, признаки

Генеральная совокупность - все объекты, о которых хотим сделать вывод. Выборка - наблюдаемые объекты. Объект - строка таблицы. Признак - столбец, используемый для описания объекта. Целевая переменная - то, что нужно предсказывать.

Типы переменных:

Тип	Примеры	Что применять
Количественная непрерывная	цена, время, доход	среднее, дисперсия, корреляция, регрессия
Количественная дискретная	число покупок, число ошибок	счетчики, регрессия/классификация по контексту

Тип	Примеры	Что применять
Категориальная номинальная	город, цвет, тип	one-hot, label encoding для деревьев
Категориальная порядковая	уровень образования, рейтинг	ordinal encoding, осторожная интерпретация порядка
Бинарная	да/нет, 0/1	классификация, доли

3.2 Меры центра и разброса

Среднее:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Медиана - значение, делящее упорядоченную выборку пополам. Медиана устойчивее среднего к выбросам.

Мода - наиболее частое значение. Для плотности распределения мода соответствует максимуму PDF.

Выборочная дисперсия:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

IQR:

$$IQR = Q_3 - Q_1.$$

Типовое правило выбросов для boxplot:

$$x < Q_1 - 1.5IQR \quad \text{или} \quad x > Q_3 + 1.5IQR.$$

3.3 Корреляция Пирсона и Спирмена

Коэффициент корреляции лежит от -1 до 1.

- $r > 0$: при росте x значение y в среднем растет.
- $r < 0$: при росте x значение y в среднем уменьшается.
- $r \approx 0$: линейной зависимости почти нет, но нелинейная зависимость возможна.

Пирсон измеряет линейную зависимость. Спирмен измеряет монотонную зависимость, потому что считает по рангам. Если в задаче явно сказано «корреляция Спирмена», не нужно автоматически трактовать ее квадрат как R^2 линейной регрессии.

Сила связи по модулю обычно интерпретируется так:

$ r $	Интерпретация
0.0-0.2	очень слабая или слабая связь
0.2-0.4	слабая/умеренная
0.4-0.7	заметная
0.7-1.0	сильная

Корреляция не доказывает причинность. Большая выборка может сделать статистически значимой даже маленькую корреляцию, но это не делает связь практически сильной.

3.4 Нулевая гипотеза, p-value, ошибки

Нулевая гипотеза H_0 - базовое предположение, обычно «эффекта нет», «разницы нет», «метод не влияет». Альтернатива H_1 - то, что исследователь хочет обнаружить.

Уровень значимости α - допустимая вероятность ошибки первого рода. Часто $\alpha = 0.05$.

Правило:

- если $p\text{-value} \leq \alpha$, отвергаем H_0 ;
- если $p\text{-value} > \alpha$, не отвергаем H_0 .

Не отвергнуть H_0 не значит доказать H_0 . Это значит, что данных недостаточно для уверенного отказа от нее.

Ошибки:

Решение	Реальность	Ошибка
Отвергли H_0	H_0 верна	Ошибка I рода, false positive
Не отвергли H_0	H_0 ложна	Ошибка II рода, false negative

Мощность критерия:

$$Power = 1 - \beta,$$

где β - вероятность ошибки второго рода.

4 Визуализация

4.1 Выбор графика

Цель	Подходящий график
Распределение числовой переменной	histogram, density plot, boxplot, violinplot
Связь двух числовых переменных	scatter plot, bubble plot
Сравнение категорий	bar chart, boxplot по группам
Временной ряд	line plot
Доли в составе	stacked bar, pie chart только для очень малого числа категорий
Матрица корреляций	heatmap

4.2 Boxplot и violinplot

Boxplot показывает медиану, квартили, межквартильный размах и потенциальные выбросы. По boxplot удобно сравнивать группы по центру, разбросу и выбросам.

Violinplot показывает форму распределения через оценку плотности. Он полезен, когда нужно видеть не только квартили, но и форму распределения: симметрию, асимметрию, несколько пиков.

4.3 Bubble plot

Bubble plot - scatter plot, где размер точки кодирует третью числовую переменную. Его используют, когда нужно показать связь двух переменных и одновременно величину третьей.

5 Работа с данными в pandas

5.1 Первичный осмотр

```
import pandas as pd
```



```

import numpy as np

df = pd.read_csv("data.csv")

df.head()
df.shape
df.info()
df.describe()
df.describe(include="all")
df.columns
df.dtypes

Пропуски:
df.isna().sum()
df.isna().mean().sort_values(ascending=False)

Уникальные значения и частоты:
df["col"].nunique()
df["col"].value_counts(dropna=False)

```

5.2 Фильтрация и выбор столбцов

```

# один столбец
df["age"]

# несколько столбцов
df[["age", "income"]]

# строки по условию
df[df["age"] >= 18]

# несколько условий: нужны скобки
df[(df["age"] >= 18) & (df["city"] == "Moscow")]

# или
df[(df["city"] == "Moscow") | (df["city"] == "SPb")]

```

5.3 Пропуски

```

# количество пропусков по столбцам
missing = df.isna().sum()

# столбец с максимумом пропусков
col_max = missing.idxmax()

# среднее по числовому столбцу, NaN игнорируются
mean_value = df[col_max].mean()

# заполнить пропуски средним
df[col_max] = df[col_max].fillna(mean_value)

# столбец с минимальным ненулевым числом пропусков
missing_nonzero = missing[missing > 0]
col_min = missing_nonzero.idxmin()

# удалить строки с пропусками в конкретном столбце
rows_deleted = df[col_min].isna().sum()
df = df.dropna(subset=[col_min])

Для категориального столбца часто заполняют модой:
mode_value = df["cat_col"].mode()[0]

```

```
df["cat_col"] = df["cat_col"].fillna(mode_value)
```

5.4 Создание и изменение столбцов

```
df["income_per_age"] = df["income"] / df["age"]
df["is_adult"] = (df["age"] >= 18).astype(int)
df["log_income"] = np.log1p(df["income"])
df["city"] = df["city"].replace({"Msk": "Moscow"})
```

Типы:

```
df["col"] = pd.to_numeric(df["col"], errors="coerce")
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df["year"] = df["date"].dt.year
```

5.5 Агрегации и сводные таблицы

```
# среднее по группам
df.groupby("city")["income"].mean()

# несколько агрегатов
df.groupby("city")["income"].agg(["count", "mean", "median", "std"])

# несколько столбцов
df.groupby("city").agg({"income": "mean", "age": "median"})

# сводная таблица
pd.pivot_table(
    df,
    values="income",
    index="city",
    columns="gender",
    aggfunc="mean"
)
```

5.6 Корреляции

```
# матрица корреляций по числовым столбцам
df.corr(numeric_only=True)

# корреляция двух столбцов
df["x"].corr(df["y"])

# корреляция Спирмена
df["x"].corr(df["y"], method="spearman")
```

5.7 Merge, concat, сортировка

```
# объединение по ключу
df = df_left.merge(df_right, on="id", how="left")

# склейка строк
all_df = pd.concat([df_train, df_test], axis=0)

# сортировка
df.sort_values("income", ascending=False)
```

6 Кодирование категориальных признаков

6.1 Label encoding

Label encoding заменяет категории числами 0, 1, 2, ... Важно: количество столбцов не увеличивается. Метод часто подходит для деревьев и бустинга, но может быть опасен для линейных моделей, потому что модель может воспринять коды как порядок.

Кодировать train и test нужно согласованно:

```
cat_cols = df_train.select_dtypes(include=["object", "category"]).columns
```

```
for col in cat_cols:
    all_values = pd.concat([df_train[col], df_test[col]], axis=0)
    codes, uniques = pd.factorize(all_values, sort=True)
    df_train[col] = codes[:len(df_train)]
    df_test[col] = codes[len(df_train):]
```

После кодировки число столбцов:

```
df_test.shape[1]
```

6.2 One-hot encoding

One-hot encoding создает отдельный бинарный столбец для каждой категории. Число столбцов увеличивается.

```
X = pd.get_dummies(X, drop_first=False)
```

В sklearn:

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
preprocess = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
        ("num", "passthrough", num_cols),
    ]
)
```

7 Машинное обучение: базовая схема

7.1 Объекты, признаки, цель

Объект - строка таблицы. Признаки - входные столбцы X . Целевая переменная - то, что предсказываем, y .

Если предсказываем количество, цену, длительность - это регрессия. Если предсказываем класс - классификация. Если меток нет и нужно найти группы - кластеризация.

7.2 Train/test split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y # для классификации, если нужно сохранить баланс классов
)
```

Нельзя подбирать гиперпараметры по тестовой выборке. Тест нужен для финальной оценки.

7.3 Pipeline

Pipeline уменьшает риск утечки данных: все преобразования обучаются только на train внутри cross-validation.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

8 Метрики регрессии

Пусть y_i - истинное значение, $\hat{y}_i$ - предсказание.

8.1 MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE измеряется в единицах целевой переменной. Менее чувствительна к выбросам, чем MSE/RMSE. Хорошо отражает типичную абсолютную ошибку.

8.2 MSE и RMSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

$$RMSE = \sqrt{MSE}.$$

MSE/RMSE сильно штрафуют крупные ошибки. Если в данных есть редкие дорогие объекты или большие выбросы, эти метрики могут в основном отражать ошибки на них.

Важно: RMSE не игнорирует крупные ошибки, а наоборот усиливает их влияние.

8.3 MAPE и MSLE

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

MAPE плохо работает, если истинные значения близки к нулю: даже небольшая абсолютная ошибка может дать огромную процентную ошибку.

MSLE:

$$MSLE = \frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2.$$

MSLE подходит для положительных величин и слабее штрафует ошибки на больших абсолютных значениях, чем MSE.

8.4 R^2

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

$R^2 = 1$ - идеальное качество. $R^2 = 0$ - не лучше предсказания среднего. $R^2 < 0$ - хуже предсказания среднего.

9 Метрики классификации

9.1 Confusion matrix

Для бинарной классификации:

	Предсказан +	Предсказан -
Истинный +	TP	FN
Истинный -	FP	TN

- TP: больной назван больным / мошенник найден.
- FN: положительный объект пропущен.
- FP: отрицательный объект ошибочно назван положительным.
- TN: отрицательный объект правильно распознан.

9.2 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Accuracy подходит, если классы достаточно сбалансированы и ошибки примерно равнозначны. При сильном дисбалансе accuracy может быть обманчивой: модель может всегда предсказывать самый частый класс и получать высокую accuracy.

9.3 Precision, recall, F1

$$Precision = \frac{TP}{TP + FP}.$$

Precision отвечает: среди объектов, которые модель назвала положительными, какая доля действительно положительная.

$$Recall = \frac{TP}{TP + FN}.$$

Recall отвечает: какую долю всех положительных объектов модель нашла.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}.$$

F1 полезен, когда важны и FP, и FN. Если задача медицинская и нельзя пропускать больных, recall обычно особенно важен. Если ложная тревога дорогая, важен precision.

Если известно F1 и сумма $FP + FN$, можно использовать формулу:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

и выразить TP алгебраически.

9.4 ROC-AUC

ROC-AUC оценивает способность модели ранжировать положительные объекты выше отрицательных. Вероятностная интерпретация:

$$ROC-AUC = P(score(x_+) > score(x_-)) + \frac{1}{2}P(score(x_+) = score(x_-)).$$

Значение около 0.5 соответствует случайному ранжированию. Чем ближе к 1, тем лучше разделимость классов.

ROC-AUC не зависит от конкретного выбранного порога классификации. Она показывает качество ранжирования, а не precision/recall при конкретном пороге.

Если все объекты получают один и тот же score, ROC-AUC равна 0.5, потому что все пары связаны ничьей.

При сильном дисбалансе классов ROC-AUC иногда выглядит слишком оптимистично; полезно дополнительно смотреть PR-AUC.

9.5 PR-AUC

PR-AUC - площадь под precision-recall кривой. Особенно полезна при редком положительном классе: мошенничество, дефекты, редкие болезни.

9.6 Выбор метрики по ситуации

Ситуация	Подходящие метрики
Классы сбалансированы, ошибки похожи по цене	accuracy, F1
Положительный класс редкий	recall, precision, F1, PR-AUC
Нужно ранжирование вероятностей	ROC-AUC, PR-AUC
Нельзя пропускать положительные объекты	recall, F1, PR-AUC
Нельзя часто ошибочно тревожить	precision, F1

10 Переобучение, недообучение, регуляризация

10.1 Недообучение

Недообучение: модель слишком простая и плохо работает даже на train. Обычно и train, и test качество низкое.

Причины: слишком простая модель, мало признаков, слишком сильная регуляризация, плохие признаки, ошибка в данных.

10.2 Переобучение

Переобучение: train качество заметно лучше test качества. Модель запомнила шум и частные особенности train.

Борьба:

- упростить модель;
- добавить регуляризацию;
- собрать больше данных;
- использовать cross-validation;
- удалить утечки;
- для деревьев ограничить глубину, минимальный размер листа;
- для нейросетей использовать dropout, weight decay, early stopping.

10.3 Когда нельзя сделать вывод

По одному числу метрики нельзя всегда сказать, хороша модель или плоха. Нужен контекст: баланс классов, baseline, цена ошибок, метрика, train/test gap.

В задачах с редким положительным классом accuracy часто плохой ориентир. Надо смотреть confusion matrix, recall, precision, F1, ROC-AUC, PR-AUC.

10.4 Регуляризация L1/L2

Линейная регрессия с L2-регуляризацией - Ridge:

$$Q(w) = Loss(w) + \lambda \sum_j w_j^2.$$

Линейная/логистическая модель с L1-регуляризацией - Lasso:

$$Q(w) = Loss(w) + \lambda \sum_j |w_j|.$$

L1 может занулять часть коэффициентов и работать как отбор признаков. L2 обычно уменьшает веса, но редко зануляет их строго.

Регуляризация чувствительна к масштабу признаков. Если признаки имеют разные масштабы, признаки с большими численными значениями могут подавлять остальные. Для линейных моделей с регуляризацией обычно нужно масштабирование.

Правильная схема:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(penalty="l1", solver="liblinear"))
])
```

11 Кросс-валидация и подбор гиперпараметров

Кросс-валидация делит train на несколько фолдов. Модель несколько раз обучается на части фолдов и проверяется на оставшемся. Это дает более устойчивую оценку качества, чем один validation split.

GridSearchCV перебирает сетку параметров:

```
from sklearn.model_selection import GridSearchCV

params = {
    "model__C": [0.01, 0.1, 1, 10],
    "model__penalty": ["l1", "l2"]
}

grid = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="f1",
    cv=5,
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_
```

best_params = grid.best_params_

Для Ridge/Lasso/логистической регрессии гиперпараметр регуляризации подбирают по cross-validation, а не по test.

12 Линейная алгебра и ядра

12.1 Векторы, нормы, скалярное произведение

Скалярное произведение:

$$a \cdot b = \sum_{i=1}^d a_i b_i.$$

Евклидова норма:

$$\|a\| = \sqrt{\sum_{i=1}^d a_i^2}.$$

Косинус угла между векторами:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}.$$

12.2 Kernel trick

Ядро $K(a, b)$ задает скалярное произведение в новом признаковом пространстве:

$$K(a, b) = \langle \varphi(a), \varphi(b) \rangle.$$

Косинус угла в новом пространстве:

$$\cos(\varphi(a), \varphi(b)) = \frac{K(a, b)}{\sqrt{K(a, a)K(b, b)}}.$$

Для RBF-подобного ядра:

$$K(a, b) = \exp(-\|a - b\|^2),$$

всегда $K(a, a) = 1$ и $K(b, b) = 1$, потому что $\|a - a\| = 0$.

13 Линейные модели и градиентный спуск

13.1 Линейная и логистическая регрессия

Линейная регрессия:

$$\hat{y} = w_0 + w_1 x_1 + \dots + w_d x_d.$$

Логистическая регрессия для бинарной классификации:

$$P(y = 1 \mid x) = \sigma(w^T x),$$

где

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Логистическая регрессия выдает вероятность, затем по порогу получается класс.

13.2 Градиентный спуск

Общая формула:

$$w_{k+1} = w_k - \eta \nabla Q(w_k),$$

где η - learning rate, $Q(w)$ - функция потерь.

Если η слишком маленькая, обучение идет медленно. Если η слишком большая, алгоритм может расходиться или «скакать» вокруг минимума.

Batch gradient descent считает градиент по всей выборке. Stochastic gradient descent, SGD, обновляет параметры по одному объекту или mini-batch. Mini-batch SGD - стандартный практический вариант.

Для функции одной переменной $L(w)$ два шага считаются так:

1. Найти производную $L'(w)$.
2. Подставить w_0 в $w_1 = w_0 - \eta L'(w_0)$.
3. Подставить w_1 в $w_2 = w_1 - \eta L'(w_1)$.

Сходимость к глобальному минимуму гарантируется не всегда. Для выпуклых задач с корректным шагом ситуация проще. Для невыпуклых задач возможны локальные минимумы, плато и нестабильность.

13.3 Масштабирование и градиентный спуск

Если признаки имеют разные масштабы, поверхность функции потерь может быть вытянутой. Градиентный спуск тогда сходится медленнее и менее стабильно. Поэтому для линейных моделей, SVM, kNN, нейросетей и регуляризации часто применяют StandardScaler.

from sklearn.preprocessing **import** StandardScaler

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

13.4 Momentum и AdaGrad

Momentum сглаживает направление движения:

$$v_{k+1} = \rho v_k + \nabla Q(w_k),$$

$$w_{k+1} = w_k - \eta v_{k+1}.$$

AdaGrad задает разные эффективные шаги для разных координат:

$$v_k = v_{k-1} + (\nabla Q(w_k))^2,$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{v_k + \varepsilon}} \nabla Q(w_k).$$

Такой метод уменьшает шаг по координатам, где градиенты часто большие.

14 Деревья решений и ансамбли

14.1 Gini impurity

Для бинарной классификации с долями классов p_0 и p_1 критерий Джини:

$$H(R) = \sum_{k=1}^2 p_k(1 - p_k).$$

Эквивалентно:

$$H(R) = 1 - p_0^2 - p_1^2.$$

Если в вершине все объекты одного класса, $H(R) = 0$. Чем больше смешанность классов, тем выше impurity.

14.2 Information Gain

Для разбиения вершины R на левую R_l и правую R_r :

$$Q = H(R) - \frac{|R_l|}{|R|}H(R_l) - \frac{|R_r|}{|R|}H(R_r).$$

Шаги расчета:

1. Посчитать доли классов в родительской вершине и $H(R)$.
2. Посчитать доли классов в левой вершине и $H(R_l)$.
3. Посчитать доли классов в правой вершине и $H(R_r)$.
4. Подставить веса $|R_l|/|R|$ и $|R_r|/|R|$.

Дерево выбирает разбиение, которое максимально уменьшает impurity, то есть дает большой Information Gain.

14.3 Random Forest

Random Forest обучает много деревьев на разных bootstrap-подвыборках и случайных подмножествах признаков. Итоговое предсказание получается усреднением или голосованием.

Эффект леса: уменьшает variance по сравнению с одним деревом. Bias обычно меняется слабее. Шум данных не меняется, потому что он связан не с моделью, а с природой данных.

14.4 Gradient Boosting

Boosting строит последовательность слабых моделей. Каждая следующая модель старается исправить ошибки ансамбля из предыдущих моделей.

В градиентном бустинге новая модель приближает отрицательный градиент функции потерь по предсказаниям. Часто базовыми моделями являются деревья, но в теории базовой моделью может быть и другой алгоритм, если он подходит для аппроксимации нужного сигнала.

Общая идея:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

где h_m - новая слабая модель, η - learning rate.

15 Bias-Variance-Noise

Разложение ошибки:

$$Err = Bias^2 + Var + \delta^2.$$

$Bias^2$ - систематическая ошибка из-за ограниченности модели. Высокий bias означает, что модель слишком простая и недообучается.

Var - чувствительность модели к конкретной обучающей выборке. Высокий variance означает риск переобучения.

δ^2 - шум в данных, неустранимая случайная часть. Изменение модели не меняет шум.

Типовые эффекты:

Изменение	Bias	Variance	Noise
Усложнить модель	обычно уменьшается	обычно увеличивается	не меняется
Добавить полиномиальные признаки	уменьшается	увеличивается	не меняется
Усилить регуляризацию	увеличивается	уменьшается	не меняется
Добавить деревья в Random Forest	почти не меняется	уменьшается	не меняется
Добавить полезный информативный признак	может уменьшиться	может слегка вырасти или почти не измениться	не меняется
Добавить шумовой признак	почти не улучшает bias	может увеличить variance	не меняется

Если новый признак слабо коррелирует со старыми, но сильно связан с целевой переменной, он несет новую информацию. Модель может лучше приближать целевую переменную, поэтому bias часто уменьшается. Разброс может не измениться сильно, если признак стабилен и модель не слишком гибкая, но в целом дополнительные признаки могут увеличивать variance.

16 Кластеризация и аномалии

16.1 K-means

K-means ищет K кластеров и минимизирует сумму квадратов расстояний до центров:

$$\sum_{i=1}^n \|x_i - c_{z_i}\|^2.$$

Алгоритм:

1. Инициализировать K центров.
2. Назначить каждую точку ближайшему центру.
3. Пересчитать центр как среднее координат точек кластера.
4. Повторять до стабилизации.

Свойства:

- чувствителен к масштабу признаков;
- чувствителен к начальной инициализации;
- требует заранее выбрать K ;
- лучше работает на компактных, примерно шарообразных кластерах;
- может сходиться к локальному минимуму.

16.2 DBSCAN

DBSCAN ищет плотные области. Параметры:

- ε - радиус окрестности;

- `min_samples` - минимальное число соседей.

Типы точек:

- `core point`: достаточно соседей;
- `border point`: сосед `core`-точки, но сама не `core`;
- `noise`: выброс.

Плюсы: находит кластеры произвольной формы и выделяет шум. Минусы: чувствителен к выбору параметров и масштабу признаков.

16.3 Агломеративная кластеризация

Агломеративная иерархическая кластеризация начинается с отдельных объектов и последовательно объединяет ближайшие объекты/кластеры.

Варианты `linkage`:

- `single`: расстояние между ближайшими точками;
- `complete`: расстояние между самыми дальними точками;
- `average`: среднее расстояние;
- `ward`: минимизация роста внутрикластерной дисперсии.

Маркер на рисунках: маленькие группы постепенно сливаются в более крупные.

16.4 Спектральная кластеризация

Спектральная кластеризация строит граф похожести объектов, использует матрицу графа и собственные векторы, затем кластеризует объекты в новом пространстве. Часто полезна для сложных нелинейных структур.

16.5 Поиск аномалий

Поиск аномалий - задача выявления необычных объектов: ботов, мошенничества, технических сбоев, нетипичных пользовательских сессий, дефектов. Это не то же самое, что обычная классификация, если заранее нет надежных меток классов.

Методы: Isolation Forest, One-Class SVM, Local Outlier Factor, автоэнкодеры, правила на основе статистических порогов.

17 Тексты

17.1 Токенизация, стоп-слова, лемматизация

Токенизация - разбиение текста на токены: слова, подслова, знаки.

Стоп-слова - очень частые слова, часто малоинформативные: «и», «в», «на», «the». Иногда удаляются, но не всегда: для некоторых задач они важны.

Стемминг грубо отрезает окончания и оставляет основу. Лемматизация приводит слово к словарной форме.

Пример принципа:

- лемматизация: разные словоформы приводятся к нормальной форме;
- стемминг: остается грубая основа, не всегда полноценное слово.

Лемматизация обычно уменьшает словарь, снижает размерность признакового пространства и может уменьшить переобучение.

17.2 Bag of Words и TF-IDF

Bag of Words строит признаки по частотам слов.

TF-IDF:

$$tfidf(t, d) = tf(t, d) \cdot idf(t),$$

где tf - частота слова в документе, idf увеличивает вес редких слов и уменьшает вес частых.

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.9
)
X_text = vectorizer.fit_transform(texts_train)

```

17.3 Снижение размерности текстовых признаков

Полная one-hot/BoW-векторизация может дать десятки тысяч признаков. Иногда полезно заменить или дополнить ее агрегатными признаками:

- длина текста;
- число слов;
- средняя длина слова;
- доля коротких/средних/длинных слов;
- доля слов из заранее заданного словаря тональности;
- число восклицательных знаков;
- доля заглавных букв.

Хороший агрегатный признак должен быть связан с задачей. Случайные признаки вроде первой буквы первого слова обычно несут мало устойчивой информации.

18 Изображения

Черно-белое изображение можно представить матрицей размера $H \times W$. Цветное RGB-изображение - тензором $H \times W \times 3$.

Пиксели часто нормализуют:

```
img = img / 255.0
```

Нормализация помогает численной стабильности и ускоряет обучение моделей.

Для классического ML изображение можно вытянуть в вектор, но при этом теряется пространственная структура. Нейросети с convolutional layers используют локальную структуру изображения.

19 Нейронные сети

Dense layer:

$$h = \phi(Wx + b).$$

Convolutional layer применяет фильтры к локальным областям изображения. Она хорошо подходит для изображений, потому что учитывает локальные паттерны и разделяет веса.

Pooling уменьшает пространственный размер карты признаков. Max pooling берет максимум в окне, average pooling - среднее.

Dropout случайно зануляет часть нейронов во время обучения и снижает переобучение.

Batch normalization нормализует активации внутри сети и стабилизирует обучение.

Функции активации:

Активация	Формула/смысл	Особенности
Sigmoid	$1/(1 + e^{-x})$	значения 0..1, может давать затухание градиента
Tanh	-1..1	центрирована около нуля, тоже может насыщаться

Активация	Формула/смысл	Особенности
ReLU	$\max(0, x)$	простая и частая, возможны dead neurons
Leaky ReLU	небольшой наклон при $x < 0$	уменьшает проблему dead ReLU
Softmax	нормирует logits в вероятности классов	выход многоклассовой классификации

20 Быстрые sklearn-шаблоны

20.1 Регрессия

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
```

```
X = df.drop(columns=["target"])
y = df["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestRegressor(random_state=42)
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred, squared=False)
r2 = r2_score(y_test, pred)
```

20.2 Классификация

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score
from sklearn.linear_model import LogisticRegression
```

```
X = df.drop(columns=["target"])
y = df["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
proba = model.predict_proba(X_test)[: , 1]
```

```
accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred)
recall = recall_score(y_test, pred)
f1 = f1_score(y_test, pred)
roc_auc = roc_auc_score(y_test, proba)
```

20.3 Pipeline с preprocessing

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

num_cols = X.select_dtypes(include=["int64", "float64"]).columns
cat_cols = X.select_dtypes(include=["object", "category"]).columns

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
])

pipe = Pipeline([
    ("prep", preprocess),
    ("model", LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train)
```

21 Экзаменационные правила-ловушки

1. Если предсказывается число - это регрессия, даже если число дискретное.
2. Если предсказывается метка класса - классификация.
3. Если нет целевой переменной - обучение без учителя.
4. Ассигу нехья слепо использовать при сильном дисбалансе классов.
5. ROC-AUC оценивает ранжирование, а не качество при одном пороге.
6. PR-AUC полезна при редком положительном классе.
7. MAPE опасна при истинных значениях около нуля.
8. RMSE усиливает влияние крупных ошибок.
9. Линейные модели с регуляризацией требуют масштабирования признаков.
10. K-means требует заранее выбрать K и чувствителен к масштабу.
11. DBSCAN может находить выбросы и кластеры произвольной формы.
12. Лемматизация уменьшает число словоформ; стемминг оставляет грубую основу.
13. CDF не убывает и лежит от 0 до 1; PDF может убывать и иметь пики.
14. Добавление сложности обычно уменьшает bias и увеличивает variance.
15. Шум данных не уменьшается от смены модели.
16. Большой learning rate может привести к расходимости градиентного спуска.
17. SGD обновляет параметры по объектам/батчам; обычный batch GD считает градиент по всей выборке.
18. Для гипотез: отвергли верную H_0 - ошибка I рода; не отвергли ложную H_0 - ошибка II рода.
19. Spearman - про монотонную зависимость, Pearson - про линейную.
20. Information Gain в деревьях - уменьшение impurity после разбиения.

22 Краткий словарь

Термин	Смысл
Feature	признак, столбец входных данных
Target	целевая переменная, ответ
Train	данные для обучения
Test	данные для финальной проверки
Validation	данные для настройки модели
Leakage	утечка информации из будущего/test в train
Baseline	простая модель для сравнения
Hyperparameter	параметр, задаваемый до обучения

Термин	Смысл
Parameter	вес модели, обучается по данным
Regularization	штраф за сложность модели
Overfitting	переобучение
Underfitting	недообучение
Bias	систематическая ошибка модели
Variance	чувствительность модели к выборке
Noise	неустраняемая случайность в данных
Token	элемент текста после токенизации
Lemma	нормальная форма слова
Embedding	векторное представление объекта
Kernel	функция скалярного произведения в новом пространстве